

UNIVERSITY OF BOLOGNA

Master's degree in Mathematics

**MLP for Image Classification
and
Surface Fitting**

Author:

Niccolò Chiari

ACADEMIC YEAR 2024/2025

Contents

1	MLP for Image Classification	1
1.1	Model Evaluation Based on Number of Dense Layers	2
1.2	Neural Comparison Analysis	5
1.3	Comparative Analysis of Activation Function	8
1.4	Batch Size Evaluation	11
1.5	Training Epochs Analysis	14
1.6	SGD vs ADAM Performance Evaluation	17
1.7	Final Considerations	19
2	INR for 3D surface fitting	21
2.1	Periodic Activation for INR	22
2.2	Initilialization and Distribution of Activations	22
2.3	Experiment	25

Chapter 1

MLP for Image Classification

The multilayer perceptron (MLP) is a feedforward neural network (FFNN), the information flows only in one direction from input to output. The architecture of MLP is defined by one input layer, one or more hidden layers, and one output layer.

One of the peculiarities of the MLPs is that are fully connected, every neuron in each layer is connected to each neuron in the next layer. Typically the neurons use non linear activation functions, allowing the network to learn complex pattern in data passed in input.

The dataset we are going to use for image classification problem is fashion - MNIST, it is a dataset of Zalando's article images divided into 60'000 images for training set and 10'000 for the test set. Each sample is a 28 x 28 grayscale image, labeled as follows:

- 0 T-shirt top;
- 1: Trouser
- 2: Pullover;
- 3: Dress
- 4: Coat;
- 5: Sandal
- 6: Shirt;
- 7: Sneaker
- 8: Bag;
- 9: Ankle boot

Examples of these images are presented below:

The MLP model will be characterized by the fundamental hyperparameters such as the number of hidden layers, neurons in each layer, activation function, batch size, optimizer choice, and number of epochs. The aim is to create the best model among the choice of these hyperparameters, we will start with an initial configuration and then after testing the different values for a single



Figure 1.1: Clothing items from fashion-MNIST

hyperparameter given we will arrive to the best MLP model. The criteria for the evaluation of the model are the following:

- Model complexity: Overall number of parameters of the model;
- Learning measures: Accuracy and Loss metrics, they are helpful to detect if the model is good or not, we are going to compare the training set and validation set in order to prevent overfitting and underfitting;
- Generalization measure: It indicates how good the model is on the unseen data thus on the test set, an high value correspond a good model;
- Training speed.

1.1 Model Evaluation Based on Number of Dense Layers

The following table represent the set of all the possible hyperparameter that we need to test in order to find the best possible MLP model

Parameters				
Number of Dense Layers	5	10	15	-
Neurons per Layer	32	64	128	-
Batch Size	32	128	256	512
Epochs	50	100	500	-
Optimizer	SGD	ADAM	-	-
Activation	ReLU	Sigmoid	Tanh	-

As initial configuration we decide to vary the number of hidden layers and maintain fixed the first choice for the other hyperparameters. The three different models we are going to train and evaluate could be represented in the following tables:

	Dense Layers	Neurons	Activation Function	Batch Size	Optimizer
Model	5	32	ReLU	32	SGD

	Dense Layers	Neurons	Activation Function	Batch Size	Optimizer
Model	10	32	ReLU	32	SGD

	Dense Layers	Neurons	Activation Function	Batch Size	Optimizer
Model	15	32	ReLU	32	SGD

Even before presenting any evaluation metrics result we will expect a bad behavior from each of the three models, the issue is the low number of neurons in each dense layer (32) compared to the dimension of the input layer (784, dimension of each vectorized image).

The model undergoes a form of dimensionality reduction or shrinkage, more precisely the first hidden layer acts as a bottleneck, forcing the model to learn a compressed representation of the input.

From the point of view of model complexity, a small number of parameters involves less computation during the training phase, also involves less risk of overfitting. These considerations are all related to the fact that we are working with a small dataset, in case of a larger dataset a larger number of parameters could be more appropriate.

The parameters involved for the three initial models involved are represented below

	5-dense layer	10-dense layer	15-dense layer
number of parameters	29 674	34 954	40 234

We can therefore conclude that from the complexity point of view, the 5 dense layer model is the simpler one and the best candidate for the classification.

The evaluation metric results are represented in the following table

n. of layers	trL	trA	vL	vA	teL	teA
5	32,71%	88,22%	36,47%	86,73%	40,18%	85,32%
10	33,78%	87,98%	40,14%	86,18%	44,09%	85,12%
15	72,27%	74,66%	78,03%	73,25%	82,01%	72,17%

and their graphical representation is the following

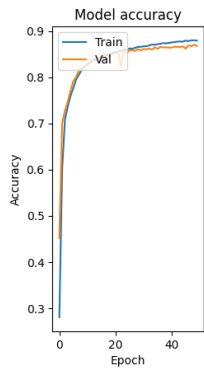


Figure 1.2: 5-dense layer

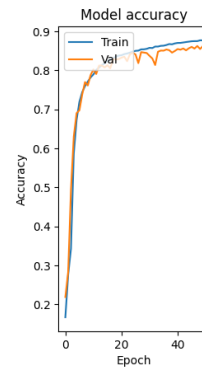
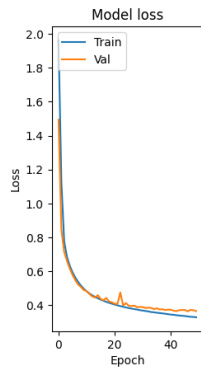


Figure 1.3: 10-dense layer

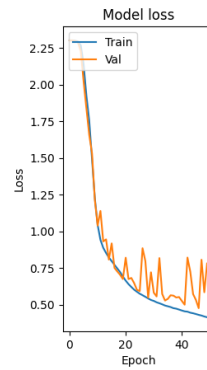
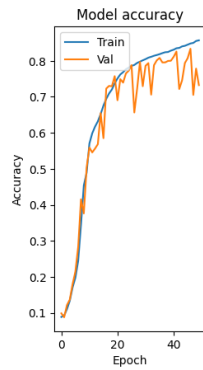
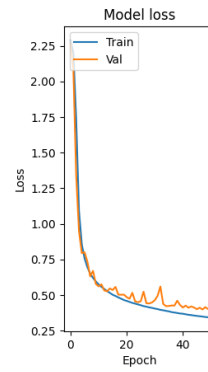


Figure 1.4: 15-dense layer

The 5-layer model performs the best, we have a good balance between training and validation loss, an high level of validation accuracy and new data are generalized well.

Regarding 10-dense layer model, we are in presence of mild overfitting, the training loss is less than the validation loss, however we have an high value for the validation accuracy and a good generalization on new data.

The 15-dense layer model shows a high loss value for all the metrics and low validation results, the model is severely underfitted.

1.2 Neural Comparation Analysis

Starting from the analysis done in the previous section, we found that the 5-dense model is the one which performs better, however we carry on also the study on the 10-layer model.

In the previous section we did an analysis based on the number of layer, now we focus on the number of neurons for each layer. We are going to expect is a better behavior associated to the maximum number of neurons possible (128).

The goal is to discard the model, given the same number of neurons, that behaves worse.

We begin analyzing the 5 dense layer model, considering different number of neurons in each layer

	Dense Layers	Neurons	Activation Function	Batch Size	Optimizer
Model	5	32	ReLU	32	SGD
	5	64	ReLU	32	SGD
	5	128	ReLU	32	SGD

Table 1.1: Configuration of 5-dense layers with different number of neurons each layer

subsequently we will do the same for the 10-dense layer model

	Dense Layers	Neurons	Activation Function	Batch Size	Optimizer
Model	10	32	ReLU	32	SGD
	10	64	ReLU	32	SGD
	10	128	ReLU	32	SGD

Table 1.2: Configuration of 10-dense layer with different number of neurons each layer

From this neural comparison analysis we will keep the sub-model that behaves better in terms of metrics result.

The results, in terms of evaluation metrics are here represented

n. of neurons	trL	trA	vL	vA	teL	teA
5-dense layer, 32 neurons	32,71%	88,22%	36,47%	86,73%	40,18%	85,32%
10-dense layer, 32 neurons	33,78%	87,98%	40,14%	86,18%	44,09%	85,12%

Table 1.3: 5-dense layer vs 10-dense layer, 32 neurons

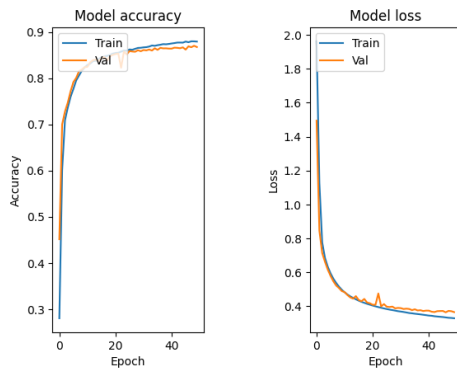


Figure 1.5: 5-dense layer, 32 neurons

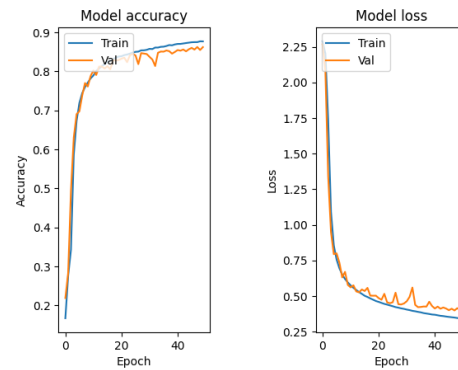


Figure 1.6: 10-dense layer, 32 neurons

5-dense layer model, 32 neurons has better behavior compared to the 10-dense layer 32 neuron, mild presence of overfitting in the second configuration.

n. of neurons	trL	trA	vL	vA	teL	teA
5-dense layer, 64 neurons	30,02%	89,08%	35,14%	87,3%	38,77%	86,12%
10-dense layer, 64 neurons	28,12%	89,77%	36,04%	87,67%	39,1%	86,41%

Table 1.4: 5-dense layer vs 10-dense layer, 64 neurons

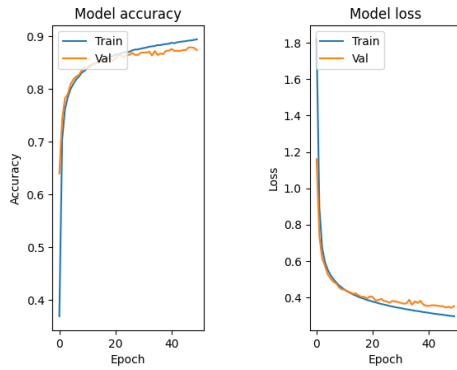


Figure 1.7: 5-dense layer, 64 neurons

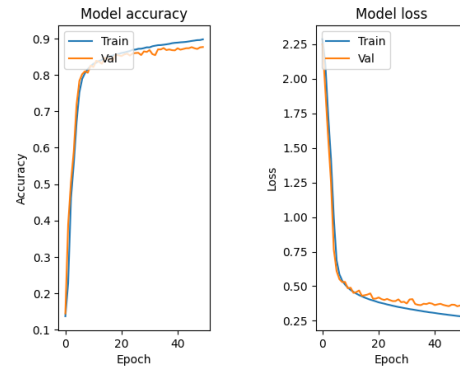


Figure 1.8: 10-dense layer, 64 neurons

10-dense layer has better improvements compared to the 5-dense layer, lower training loss and higher validation accuracy, however we have mild overfitting.

The 5-dense layer, 64 neurons is a better choice.

n. of neurons	trL	trA	vL	vA	teL	teA
5-dense layer, 128 neurons	28,97%	89,37%	35,3%	87,4%	39,71%	85,87%
10-dense layer, 128 neurons	26,6%	90,2%	36,6%	87,3%	40,9%	86,27%

Table 1.5: 5-dense layer vs 10-dense layer, 128 neurons

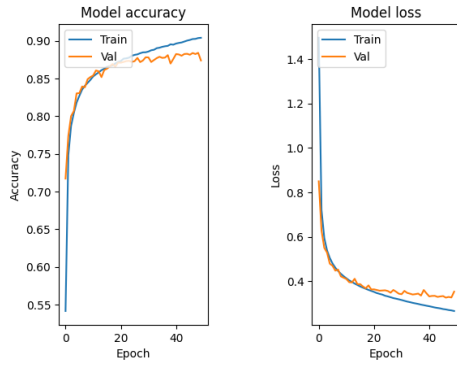


Figure 1.9: 5-dense layer, 128 neurons

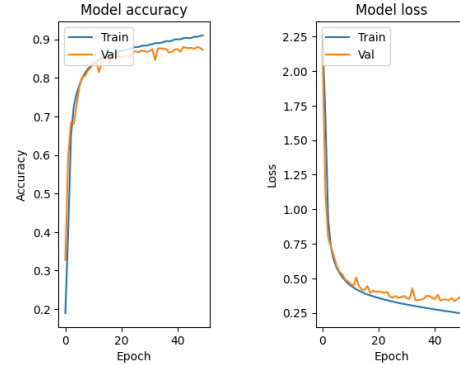


Figure 1.10: 10-dense layer, 128 neurons

Few words need to be spent on the last configuration, as the tables may suggest, the 10-layer model 128 neuron seems more appropriate, however the 5-dense layer is able to treat better new data, just like above.

1.3 Comparative Analysis of Activation Function

Until now we have found that the best models are those with 5-dense layers, in particular the model that best suits the problem from a dimensional and computational point of view is the one with 128 neurons each layer. However, we also carry on the study of the other two, the models with 64 and 32 neurons each layer, in order to have a comparison once we have found the best choices for the remaining hyperparameters.

All models have so far been tested using the ReLU activation function. The goal now is to evaluate the performance of the other activation functions introduced earlier for all of the three models, in order to determine which activation function is the most effective.

a.f.	trL	trA	vL	vA	teL	teA
ReLU	32,71%	88,22%	36,47%	86,73%	40,2%	85,33%
Sigmoid	230,14%	14,8%	230,14%	14,33%	230,14%	14,48%
Tanh	31,66%	88,89%	37,39%	86,66%	40,37%	85,54%

Table 1.6: 5-dense layer, 32 neurons

5-dense layer, 32 neurons, Sigmoid activation function is severely under-performing, no learning is happening and the loss value for all the three set is very high. For this reason, we decide to not consider the Sigmoid activation in the other two models.

The Tanh architecture has a higher training and validation accuracy, it also generalizes better. The only problem is the presence of mild overfitting but it is small enough not to cause problems.

The graphical representation is the following

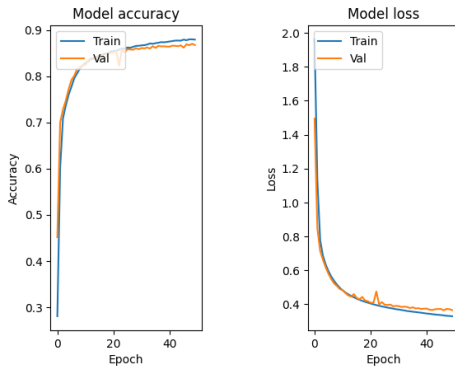


Figure 1.11: 5-dense layer, 32 neurons, ReLU a.f.

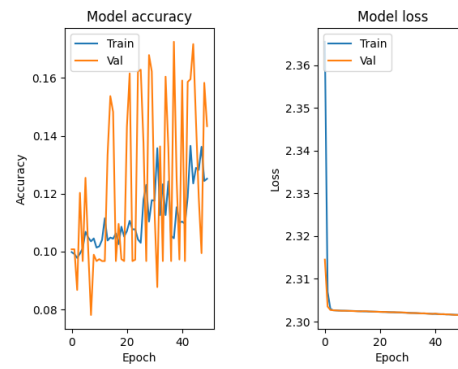


Figure 1.12: 5-dense layer, 32 neurons, Sigmoid a.f.

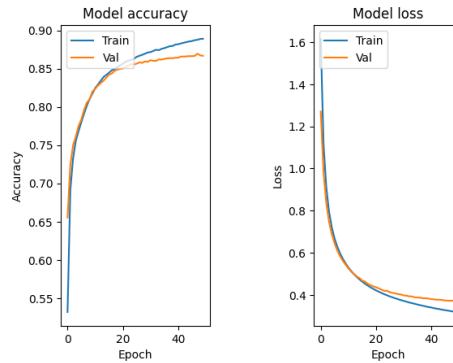


Figure 1.13: 5-dense layer, 32 neurons, Tanh a.f.

The behaviour observed for the 5-dense layer, 32 neurons, where tanh is a better choice compared to ReLU, also hold for the 64 and 128 neuron architecture.

a.f.	trL	trA	vL	vA	teL	teA
ReLU	30,02%	89,08%	35,14%	87,3%	38,77%	86,12%
Tanh	28,33%	90,14%	34,08%	87,88%	38,17%	86,55%

Table 1.7: 5-dense layer, 64 neurons

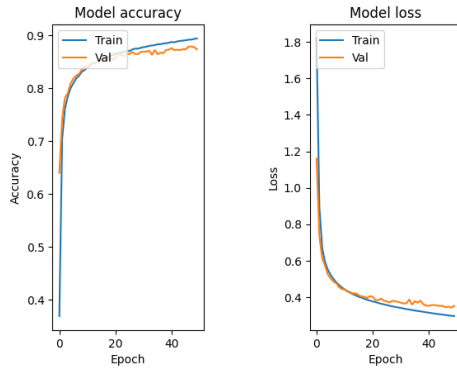


Figure 1.14: 5-dense layer, 64 neurons, ReLU a.f.

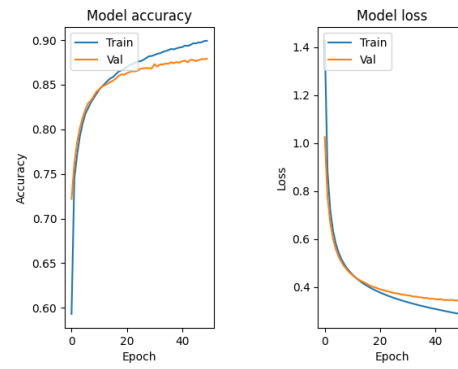


Figure 1.15: 5-dense layer, 64 neurons, Tanh a.f.

5 dense layer, 64 neurons, Tanh activation function has a better training accuracy and lower loss compared to the ReLU model, furthermore has an higher test accuracy and lower validation loss meaning it generalize better the data.

a.f.	trL	trA	vL	vA	teL	teA
ReLU	28,97%	89,37%	35,3%	87,4%	39,71%	85,87%
Tanh	25,2%	91,1%	32,6%	88,42%	36,24%	87,06%

Table 1.8: 5-dense layer, 128 neurons

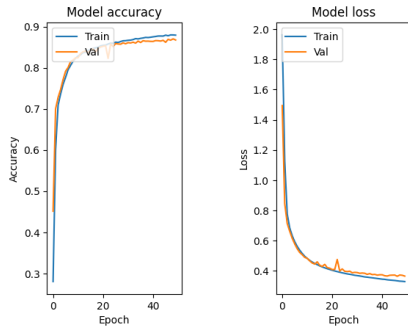


Figure 1.16: 5-dense layer, 128 neurons, ReLU a.f.

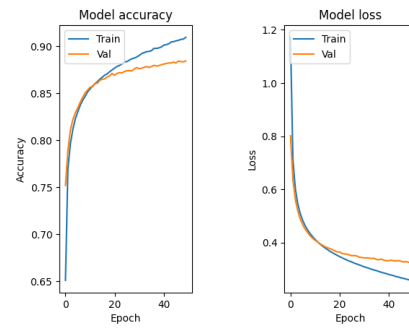


Figure 1.17: 5-dense layer, 128 neurons, Tanh a.f.

The considerations made on the 64-neuron architecture could be extended for the 128-neuron architecture.

We can conclude that the best activation function for all of the three models is Tanh, it is the one that provides an higher value both for the test accuracy and for the validation accuracy.

1.4 Batch Size Evaluation

The batch size has a significant impact on training dynamics and computational efficiency. Until now all the models have been test with a small batch size, equal to 32, this leads to a low epoch training; Our aim is to test the MLPs models with different batch sizes, in order to speed up the epochs training.

All the results will be presented through graphs and tables.

batch size	trL	trA	vL	vA	teL	teA
32bS	31,66%	88,89%	37,39%	86,66%	40,37%	85,54%
128bS	48,3%	83,68%	49,09%	83,13%	52,22%	82%
256bS	60,47%	80,29%	60,25%	80,67%	62,57%	79,5%
512bS	79,15%	75,25%	78,86%	75,50%	80,77%	74,58%

Table 1.9: 5-dense layer, 32 neurons, Tanh a.f.

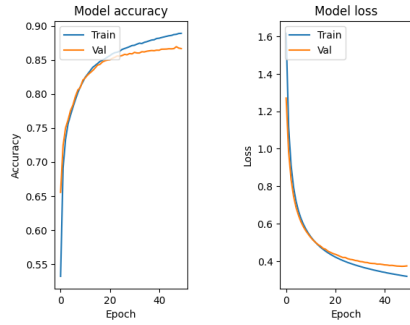


Figure 1.18: 5-dense layer, 32 neurons, Tanh a.f., 32bS

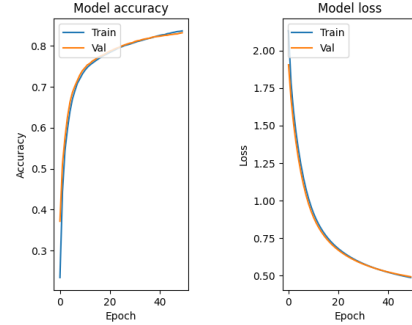


Figure 1.19: 5-dense layer, 32 neurons, Tanh a.f., 128bS

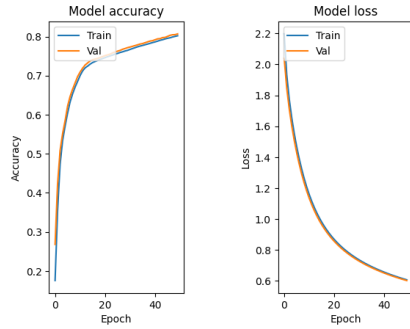


Figure 1.20: 5-dense layer, 32 neurons, Tanh a.f., 256bS

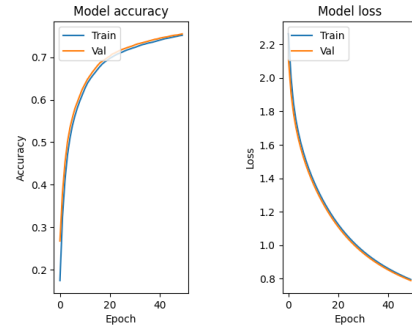


Figure 1.21: 5-dense layer, 32 neurons, Tanh a.f., 512bS

From the point of view of the accuracy, the best model is the one with 32 batch size, it has higher values on both validation and test set, however considering the stability analysis the most appropriate model is the one with 128 batch size, it has smaller gap between validation-training loss and accuracy.

batch size	trL	trA	vL	vA	teL	teA
32bS	28,33%	90,14%	34,08%	87,88%	38,17%	86,55%
128bS	41,89%	85,29%	42,84%	84,77%	46,48%	83,60%
256bS	53,64%	81,71%	53,22%	81,65%	56,38%	80,27%
512bS	64,26%	79,20%	67,87%	79,59%	66,48%	78,04%

Table 1.10: 5-dense layer, 64 neurons, Tanh a.f.

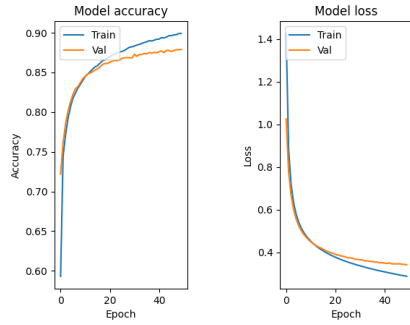


Figure 1.22: 5-dense layer, 64 neurons, Tanh a.f., 32bS

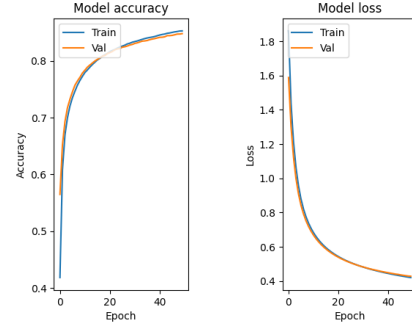


Figure 1.23: 5-dense layer, 64 neurons, Tanh a.f., 128bS

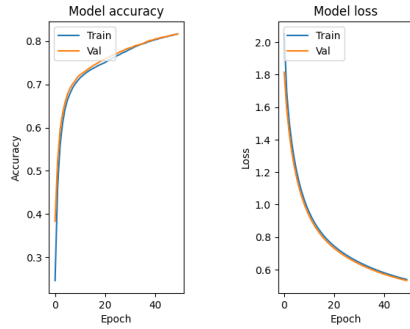


Figure 1.24: 5-dense layer, 64 neurons, Tanh a.f., 256bS

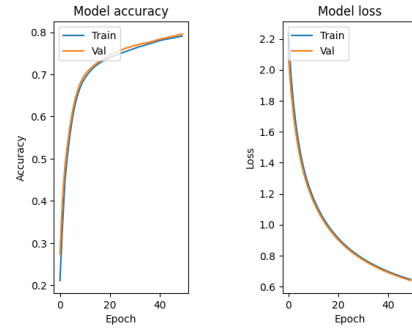


Figure 1.25: 5-dense layer, 64 neurons, Tanh a.f., 512bS

Following the previous analysis made on batch size, we can conclude that the best batch size for 5-dense layer, 64 neurons, tanh activation function is 128.

batch size	trL	trA	vL	vA	teL	teA
32bS	25,2%	91,09%	32,56%	88,42%	36,24%	87,06%
128bS	39%	86,31%	39,55%	85,77%	44%	84,1%
256bS	46,96%	83,39%	47,17%	83,25%	50,17%	82%
512bS	57,66%	79,62%	57,19%	79,77%	59,7%	78,54%

Table 1.11: 5-dense layer, 128 neurons, Tanh a.f.

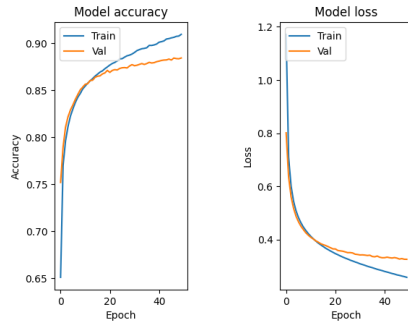


Figure 1.26: 5-dense layer, 128 neurons, Tanh a.f., 32bS

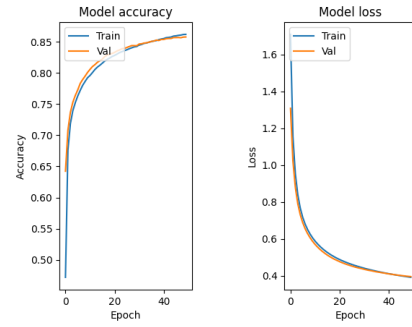


Figure 1.27: 5-dense layer, 128 neurons, Tanh a.f., 128bS

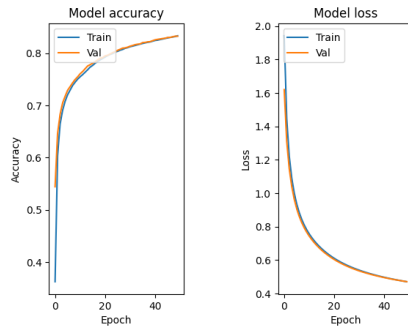


Figure 1.28: 5-dense layer, 128 neurons, Tanh a.f., 256bS

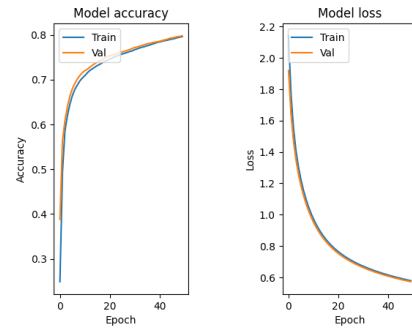


Figure 1.29: 5-dense layer, 128 neurons, Tanh a.f., 512bS

The batch size analysis for 5-dense layer, 128 neurons, Tanh activation function is identical to the previous ones.

1.5 Training Epochs Analysis

Until now, the models have been trained using a fixed number of epochs.

The aim is to see how the evaluation metrics change considering a different number of epochs for the training procedure of each model.

The results, both numerically and graphical, are presented in the following tables

n. epochs	trL	trA	vL	vA	teL	teA
50	48,3%	83,68%	49,09%	83,13%	52,22%	81,99%
100	38,18%	86,78%	39,37%	86,23%	43,8%	84,94%

Table 1.12: 5-dense layer, 32 neurons, Tanh a.f., 128 batch size

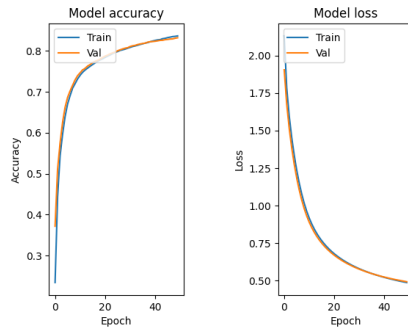


Figure 1.30: 5-dense layer, 32 neurons, Tanh, 128bS, 50 epochs

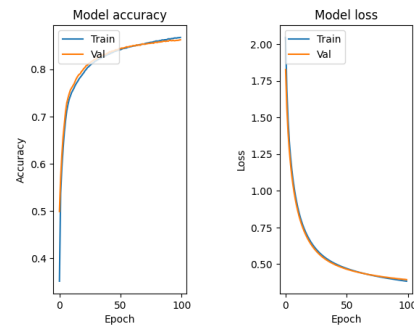


Figure 1.31: 5-dense layer, 32 neurons, Tanh, 128bS, 100 epochs

the model shows improved performance when trained with 100 epochs instead 50 epochs, the accuracy on training, validation and test is higher. 100-epochs model has mild overfitting, however we prefer this architecture thanks to the good results obtained in the accuracy.

The considerations for the other two models are almost identical.

n.epochs	trL	trA	vL	vA	teL	teA
50	41,89%	85,29%	42,83%	84,77%	46,48%	83,6%
100	34,94%	87,91%	37,49%	86,79%	41,18%	85,1%

Table 1.13: 5-dense layer, 64 neurons, Tanh a.f., 128 batch size

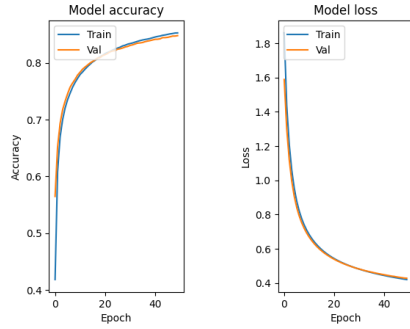


Figure 1.32: 5-dense layer, 64 neurons, Tanh, 128bS, 50 epochs

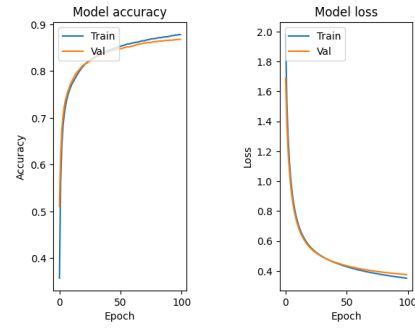


Figure 1.33: 5-dense layer, 64 neurons, Tanh, 128bS, 100 epochs

n. epochs	trL	trA	vL	vA	teL	teA
50	39%	86,31%	39,55%	85,77%	44%	84,1%
100	32,44%	88,49%	35,64%	87,33%	39,49%	85,77%

Table 1.14: 5-dense layer, 128 neurons, Tanh a.f., 128 batch size

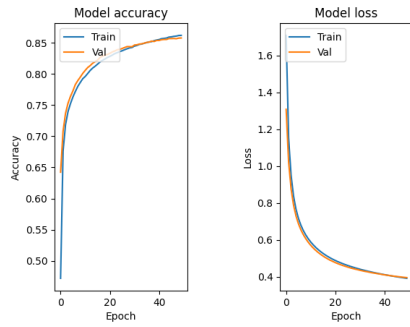


Figure 1.34: 5-dense layer, 128 neurons, Tanh, 128bS, 50 epochs

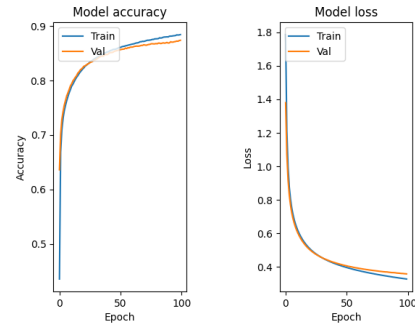


Figure 1.35: 5-dense layer, 128 neurons, Tanh, 128bS, 100 epochs

The models demonstrate improved performance metrics when trained over 100 epochs.

1.6 SGD vs ADAM Performance Evaluation

The goal is to compare the evaluation metrics using two different type of activation function , the stochastic gradient descent (SGD) optimizer and ADAM optimizer, a slightly different version of SGD based on adaptivity of first-order and second-order moments.

Optimizer	trL	trA	vL	vA	teL	teA
SGD	38,18%	86,78%	39,37%	86,23%	43,8%	84,94%
ADAM	13,4%	95,23%	55,18%	86,99%	60,14%	85,97%

Table 1.15: 5-dense layer, 32 neurons, Tanh a.f., 128 batch size, 100 epochs

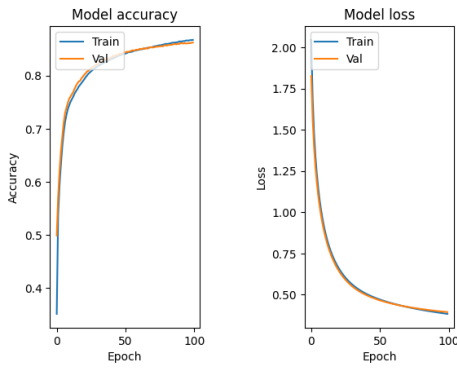


Figure 1.36: 5-dense layer, 32 neurons, Tanh a.f., 128bS, 100 epochs, SGD optimizer

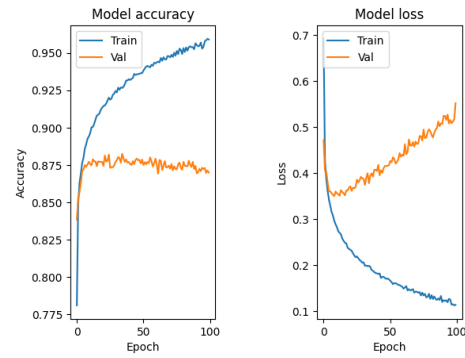


Figure 1.37: 5-dense layer, 32 neurons, Tanh a.f., 128bS, 100 epochs, ADAM optimizer

The ADAM-based model shows a lower training loss with respect to the SGD one and higher test accuracy, however, such a high value of accuracy on training is a sign of overfitting. This overfitting behavior can be observed from the fact that the training loss is much lower than the validation loss.

These considerations can also be applied to the other two models.

Optimizer	trL	trA	vL	vA	teL	teA
SGD	34,94%	87,91%	37,49%	86,79%	41,18%	85,1%
ADAM	6,4%	97,55%	57,96%	88,34%	64,42%	87,16%

Table 1.16: 5-dense layer, 64 neurons, Tanh a.f., 128 batch size, 100 epochs

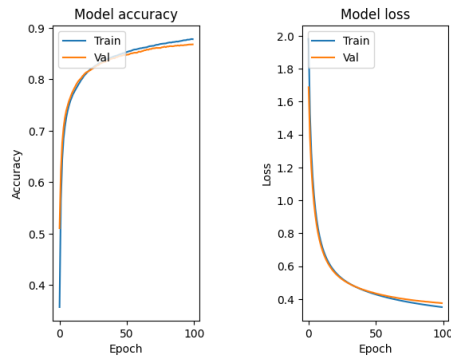


Figure 1.38: 5-dense layer, 64 neurons, Tanh a.f., 128bS, 100 epochs, SGD optimizer

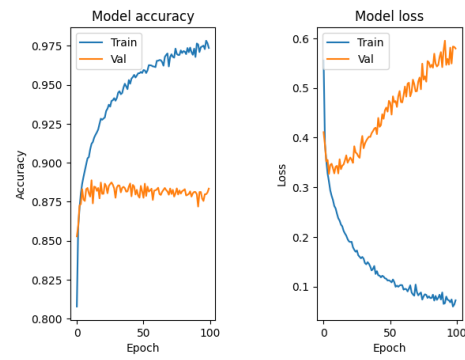


Figure 1.39: 5-dense layer, 64 neurons, Tanh a.f., 128bS, 100 epochs, ADAM optimizer

Optimizer	trL	trA	vL	vA	teL	teA
SGD	32,44%	88,49%	35,64%	87,33%	39,49%	85,77%
ADAM	5,56%	97,91%	54,18%	88,57%	59,2%	87,66%

Table 1.17: 5-dense layer, 128 neurons, Tanh a.f., 128 batch size, 100 epochs

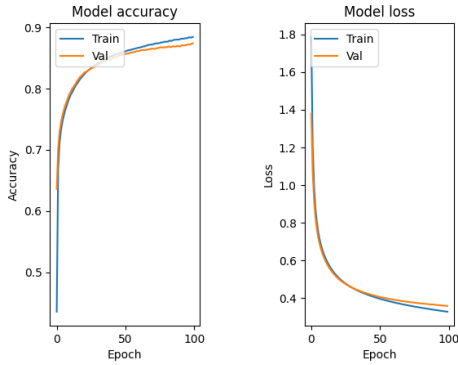


Figure 1.40: 5-dense layer, 128 neurons, Tanh a.f., 128bS, 100 epochs, SGD optimizer

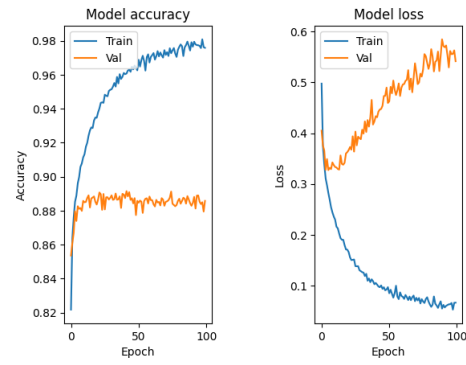


Figure 1.41: 5-dense layer, 128 neurons, Tanh a.f., 128bS, 100 epochs, ADAM optimizer

The best optimizer is the stochastic gradient descent (SGD).

1.7 Final Considerations

The following tables show the best choice of hyperparameters and the evaluation metrics of each model:

	trL	trA	vL	vA	teL	teA
Model 1	38,18%	86,78%	39,37%	86,23%	43,8%	84,94%

Table 1.18: 5-dense layer, 32 neurons, Tanh a.f., 128 batch size, 100 epochs

	trL	trA	vL	vA	teL	teA
Model 2	34,94%	87,91%	37,49%	86,79%	41,18%	85,1%

Table 1.19: 5-dense layer, 64 neurons, Tanh a.f., 128 batch size, 100 epochs

	trL	trA	vL	vA	teL	teA
Model 3	32,44%	88,49%	35,64%	87,33%	39,49%	85,77%

Table 1.20: 5-dense layer, 128 neurons, Tanh a.f., 128 batch size, 100 epochs

The best model is the third, it is the one with the lowest value for the training loss and generalize better the data with respect the other two.

Chapter 2

INR for 3D surface fitting

Multilayer perceptrons are generally used for task as classification and regression, typical activation functions are ReLu or Tanh, they take as input any signal outputs labels. The conventional signal representation is the discrete one, images are grid of pixels, audio signals are samples of amplitudes, 3D shape are point clouds or meshes.

Implicit neural representations (INR) are designed to process any input signal like an image, audio signal, point clouds and encode that into a neural network, however INR parametrize the signal as a continuous function that maps the domain of the signal (i.e. a pixel coordinate for images) to whatever is at that coordinate (for images R, G, B color).

Mathematically this can be expressed in finding Φ such that

$$F(\mathbf{x}, \Phi, \nabla_{\mathbf{x}}\Phi, \nabla_{\mathbf{x}}^2\Phi, \dots) = 0, \quad \Phi : \mathbf{x} \longrightarrow \Phi(\mathbf{x}), \quad \mathbf{x} \in \mathbb{R}^m \quad (1)$$

The goal is to learn a NN that parameterizes Φ , implicitly defined by F , in order to map $\mathbf{x}$ to some quantity of interest, depending on the problem we are studying, while satisfying (1).

However the classical network architectures, ReLU or Tanh - based fails for this implicit representation; These architectures lack the capacity to represent fine details, and they typically do not represent the derivatives of a target signal well. This is due to the fact that ReLU networks are piecewise linear, their second derivative is zero, while an activation function as Tanh is able to represent high-order derivatives but are often not well behaved, thus fails to represent details.

2.1 Periodic Activation for INR

In order to overcome this limitation, we consider periodic activation functions for implicit neural representation. The model we will use is SIREN (Sinusoidal Representation Networks), a NN that uses the sine as a periodic activation function. Its architecture follows here:

$$\Phi(\mathbf{x}) = \mathbf{W}_n(\phi_{n-1} \circ \phi_{n-2} \circ \cdots \circ \phi_0)(\mathbf{x}) + b_n \quad \mathbf{x}_l \longrightarrow \phi(\mathbf{x}_l) = \sin(\mathbf{W}_l \mathbf{x}_l + b_l)$$

where $\phi_l : \mathbb{R}^{M_l} \longrightarrow \mathbb{R}^{N_l}$ is the l -th layer, $\mathbf{W}_l$ is the weight matrix associated to the l -th layer, $\mathbf{W}_l \in \mathbb{R}^{N_l \times M_l}$, and $b_l \in \mathbb{R}^{N_l}$ bias term. The last later $(\mathbf{W}_n, b_n)$ associate the hidden features to the output.

For instance we set $M_l = M$ and $N_l = N \quad \forall$ each l -th layer.

Furthermore the training of SIREN can be accelerated by leveraging a factor ω_0 in all layers, in this way:

$$\phi_l(\mathbf{x}_l) = \sin(\omega_0 \mathbf{W}_l \mathbf{x}_l + b_l)$$

the best value for the training procedure is $\omega_0 = 30$.

2.2 Initialization and Distribution of Activations

We present the initialization scheme for the training procedure. The key idea is to preserve the distribution of activations through the layers. In this way the output of the model will not be dependent by the number of layers.

We want to prove that the activations throughout a SIREN are standard normal distributed before each sine non linearity and arcsine distributed after each sine non linearity, this will happen with a certain choice of weights for the first layer and a slightly different one for the other layers. The bias term doesn't change distribution for high enough frequency, thus we will not consider the term for the computation.

Starting from the first later, we consider as input a random variable uniformly distributed between $[-1, 1]$

$$\mathbf{x} = (x_i)_{i \leq M} \sim \mathcal{U}(-1, 1) \text{ i.i.d.}$$

and we set the matrix weights in the following way

$$\mathbf{W} = (W_{i,j})_{\substack{i \leq N \\ j \leq M}} \sim \mathcal{U}\left(-\frac{c}{\text{fan-in}}, \frac{c}{\text{fan-in}}\right) \text{ i.i.d.}$$

where c is a positive constant. In order to prove that $\omega_0 \mathbf{W}\mathbf{x}$ follows the standard normal distribution we first introduce

Lemma 2.2.1

The variance of a random variable $\mathbf{X} \sim \mathcal{U}(-a, b)$ is $\text{Var}[\mathbf{X}] = \frac{1}{12}(b - a)^2$

We use this result to compute the expected value and variance of $\omega_0 \mathbf{W}\mathbf{x}$, which are necessary for verifying its distribution.

- $\mathbb{E}[\omega_0 \mathbf{W}\mathbf{x}] = \omega_0 \mathbb{E} \left[\sum_{i=1}^N \sum_{j=1}^M w_{i,j} x_j \right] = \omega_0 \sum_{i=1}^N \sum_{j=1}^M \mathbb{E}[w_{i,j}] \mathbb{E}[x_j] = 0;$

$$\begin{aligned}
 \text{Var}[\omega_0 \mathbf{W}\mathbf{x}] &= \text{Var} \left[\omega_0 \sum_{i=1}^N \sum_{j=1}^M w_{i,j} x_j \right] = \omega_0^2 \sum_{i=1}^N \sum_{j=1}^M \text{Var}[w_{i,j} x_j] \\
 &= \omega_0^2 \sum_{i=1}^N \sum_{j=1}^M (\mathbb{E}[(w_{i,j} x_j)^2] - \mathbb{E}[w_{i,j} x_j]^2) \\
 &= \omega_0^2 \sum_{i=1}^N \sum_{j=1}^M \mathbb{E}[w_{i,j}^2] \mathbb{E}[x_j^2] = \sum_{i=1}^N \sum_{j=1}^M \frac{1}{3} \frac{c^2}{3fan - in^2} \\
 &= \frac{\omega_0^2 c^2}{9fan - in}.
 \end{aligned}$$

Choosing $c = \frac{3\sqrt{fan-in}}{\omega_0}$, applying the central limit theorem with the weak Lindeberg's condition we can assume that $\omega_0 \mathbf{W}\mathbf{x}$ follows the standard normal distribution, as can be seen in the figure below:

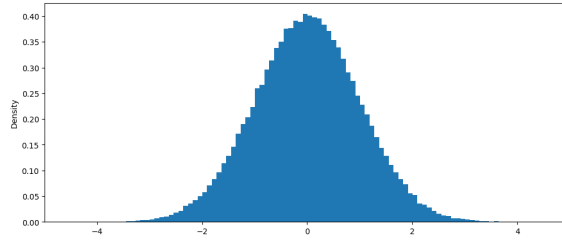


Figure 2.1: Distribution of $\omega_0 \mathbf{W}\mathbf{x}$

Therefore, the weight matrix of the first layer has the following distribution

$$\mathbf{W} \sim \mathcal{U} \left(-\frac{\sqrt{\frac{9}{fan-in}}}{\omega_0}, \frac{\sqrt{\frac{9}{fan-in}}}{\omega_0} \right).$$

The distribution of $\sin(\omega_0 \mathbf{W}\mathbf{x})$, as shown in the figure below:

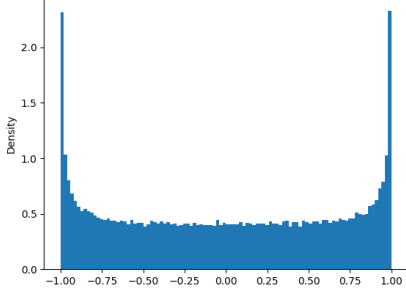


Figure 2.2: Distribution of $\sin(\omega_0 \mathbf{W}\mathbf{x})$

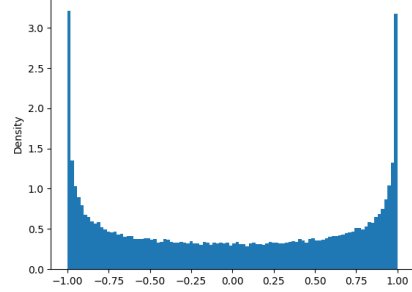


Figure 2.3: Distribution of $\arcsin(-1, 1)$

resembles the distribution of $\arcsin(-1, 1)$.

We can conclude saying that $\sin(\omega_0 \mathbf{W}\mathbf{x})$ is approximately arcsin distributed between $(-1, 1)$.

Now, we consider as input for the second layer a r.v. $\mathbf{x} \sim \arcsin(-1, 1)$ and the weights of the second layer and subsequent layers will have the following distribution $\mathbf{W} \sim \mathcal{U}(-c, c)$, c positive constant to be chosen.

As before, we want to prove that $\omega_0 \mathbf{W}\mathbf{x} \sim \mathcal{N}(0, 1)$ computing the expected value and the variance.

Lemma 2.2.2

The variance of a random variable $\mathbf{X} \sim \arcsin(-1, 1)$ is $\text{Var}[\mathbf{X}] = \frac{(b-a)^2}{8}$

$$\bullet \quad \mathbb{E}[\omega_0 \mathbf{W}\mathbf{x}] = \omega_0 \mathbb{E} \left[\sum_{i=1}^N \sum_{j=1}^M w_{i,j} x_j \right] = \omega_0 \sum_{i=1}^N \sum_{j=1}^M \mathbb{E}[w_{i,j}] \mathbb{E}[x_j] = 0$$

$$\begin{aligned} \text{Var}[\omega_0 \mathbf{W}\mathbf{x}] &= \text{Var} \left[\omega_0 \sum_{i=1}^N \sum_{j=1}^M w_{i,j} x_j \right] = \omega_0^2 \sum_{i=1}^N \sum_{j=1}^M \text{Var}[w_{i,j} x_j] \\ &= \omega_0^2 \sum_{i=1}^N \sum_{j=1}^M (\mathbb{E}[(w_{i,j} x_j)^2] - \mathbb{E}[w_{i,j} x_j]^2) \\ &\bullet \\ &= \omega_0^2 \sum_{i=1}^N \sum_{j=1}^M \mathbb{E}[w_{i,j}^2] \mathbb{E}[x_j^2] = \sum_{i=1}^N \sum_{j=1}^M \frac{1}{2} \frac{c^2}{3} \\ &= \frac{\omega_0^2 c^2 \text{ fan-in}}{6} \end{aligned}$$

Choosing $c = \sqrt{\frac{6}{f_{an-in} \omega_0^2}}$, applying the central limit theorem with the weak Lindeberg's condition we can assume that $\omega_0 \mathbf{W} \mathbf{x}$ is approximately standard normal distributed.

The weights of the second layer and subsequent layers follow this distribution: $\mathbf{W} \sim \mathcal{U}\left(-\frac{\sqrt{\frac{6}{f_{an-in}}}}{\omega_0}, \frac{\sqrt{\frac{6}{f_{an-in}}}}{\omega_0}\right)$.

As before $\sin(\omega_0 \mathbf{W} \mathbf{x}) \sim \arcsin(-1, 1)$. We proved that the distributions are preserved throughout the network.

2.3 Experiment

The goal of this report is to use SIREN (Sinusoidal Representation Networks) to reconstruct 3D shapes from a point cloud—a collection of data points in 3D space—by leveraging signed distance functions (SDFs). In this study, we specifically focus on reconstructing the shape of a bunny from its corresponding point cloud.

To do that we build a 5-layer SIREN model, each layer with 32 neurons, initializing the model as proposed in the section before. Regarding this experiment, the best value of ω_0 for the training procedure is equal to 30.

We propose three different reconstructions of the bunny, based on an increasing number of epochs:

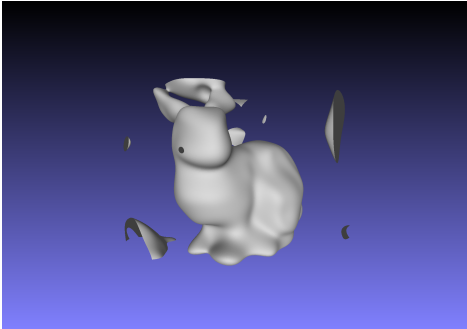


Figure 2.4: 1000 epochs

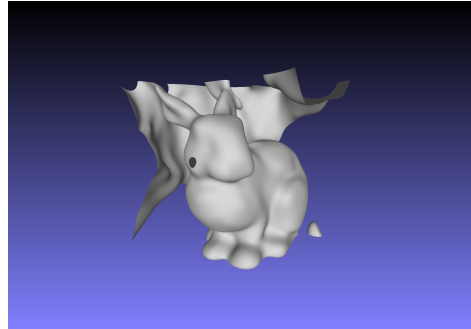


Figure 2.5: 5000 epochs

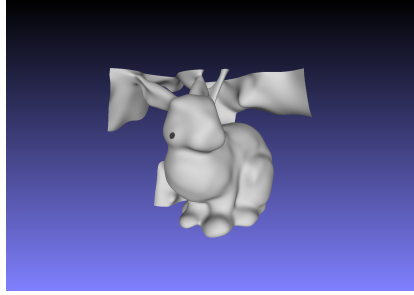


Figure 2.6: 10000 iterations

If we consider a slightly different distribution for the first layer, $\mathbf{W} \sim \mathcal{U}\left(-\frac{\sqrt{\frac{3}{fan-in}}}{\omega_0}, \frac{\sqrt{\frac{3}{fan-in}}}{\omega_0}\right)$ we have a much better rabbit reconstruction:

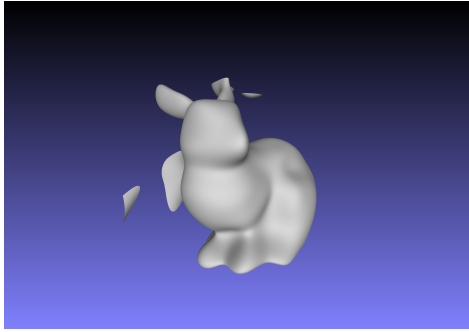


Figure 2.7: 1000 epochs

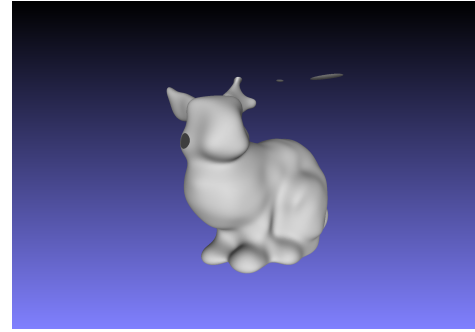


Figure 2.8: 5000 epochs

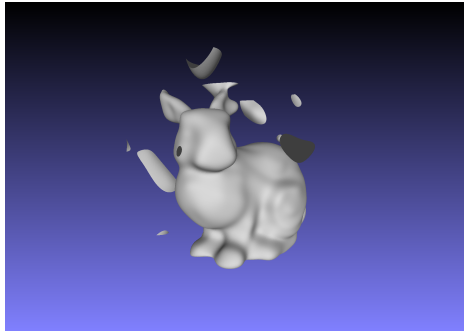


Figure 2.9: 10000 iterations

The code of SIREN neural network can be found [here](#).